seeStack — Final Project Report

> A focused, self-contained graduation-project monitoring tool. A user signs up, owns projects directly, and each project receives errors and watches websites. There are **no organizations, no memberships, no plans, no subscriptions** — just a user, their projects, and the two monitoring flows.

---

1. Project Overview

seeStack is a small monitoring platform with two flows:

1. **Error Monitoring** — an SDK POSTs exceptions to `/ingest/v1/errors` with a project-scoped API key; the backend buffers on Kafka, groups repeats by SHA-256 fingerprint, and presents them in the dashboard.
2. **Website Monitoring** — users register URLs with an interval; `MonitorScheduler` runs every 60 s, classifies each check as up or down, and surfaces status + uptime %.

Auth is email + password with a server-issued HS256 JWT. The dashboard shows exactly four pages: **Overview**, **Projects**, **Errors**, **Monitors**.

Team Responsibilities

Hi Dr., I hope you are doing well.

The responsibilities were distributed across the team as follows:

- **Abdualziz Alosaimi & Salem Omran** — Backend development and system architecture. Designed the Spring Boot modules, the internal JWT auth layer, the Kafka-backed ingest pipeline, and the PostgreSQL + ClickHouse schema.
- **Mohammed Taleb & Mohammed bin Eid** — Frontend development and dashboard implementation. Built the React + Vite SPA, including Overview, Projects, Errors, Monitors, and SDK Setup pages, and the internal email + password auth flow.
- **Omar Alallaf & Faisal Alghanim** — Testing and documentation. Wrote the runtime-validation suite (unit tests, SDK e2e tests, Chrome-MCP UI checks) and authored the Final Project Report, one-pager, and the CD package.

---

2. Data Model (after this cleanup)

```
users   ──┬──>  projects  ──┬──>  api_keys         (one per project by default)
          │                 ├──>  monitor_configs
          │                 └──>  error_groups
```

Just six tables: `users`, `projects`, `api_keys`, `monitor_configs`, `error_groups`, plus Flyway's `flyway_schema_history`. All 30+ legacy migrations were replaced with a single consolidated `V1__schema.sql`.

Removed entirely: `organizations`, `project_members`, `chat_*`, `cron_monitors`, `feature_flags`, `flag_audit_log`, `ssh_servers`, `subscriptions`, `billing_*`, `alert_rules`, `notification_log`, and every column that referenced `org_id`.

---

3. System Architecture

3.1 Backend modules — kept

```
backend/src/main/java/com/seestack/modules/
├── auth/
│   ├── controller/AuthController        POST /api/auth/register · POST /api/auth/login
│   ├── service/AuthService              BCrypt + auto-create a default project
│   ├── service/JwtService               HS256 issue/parse (zero deps)
│   └── dto/AuthDtos                     RegisterRequest · LoginRequest · AuthResponse
├── teams/
│   ├── entity/{UserEntity, ProjectEntity, ApiKeyEntity}
│   ├── repository/{UserRepository, ProjectRepository, ApiKeyManagementRepository}
│   ├── service/{ProjectService, ApiKeyGeneratorService}
│   └── controller/ProjectController     /api/v1/projects (list / create / detail / delete)
├── errors/
│   ├── controller/ErrorIngestController /ingest/v1/errors (API key)
│   ├── controller/ErrorController       /api/v1/errors (JWT)
│   ├── service/ErrorFingerprintService
│   └── service/ErrorGroupService
└── monitors/
    ├── controller/MonitorController     /api/v1/monitors (JWT)
    ├── service/MonitorService
    └── service/MonitorScheduler         @Scheduled(fixedRate = 60_000)
```

Cross-cutting: `shared/security/{SecurityConfig, JwtAuthFilter, ApiKeyAuthFilter, CurrentUser}`.

3.2 Backend modules — removed in this cleanup

- `modules/alerts`, `modules/chat`, `modules/cron`, `modules/flags`, `modules/logs`, `modules/replay`, `modules/requests`, `modules/ssh` — deleted.

- `modules/teams/`: `OrganizationController`, `OrganizationService`, `OrganizationRepository`, `OrganizationEntity`, `OrgResponse`, `OrgCreateRequest`, `OrgSettingsController`, `ProjectMemberController`, `ProjectMemberService`, `ProjectMemberRepository`, `ProjectMemberEntity`, `MemberRequest`, `MemberResponse`, `ApiKeyController`, `ApiKeyManagementController`, `ApiKeyManagementService`, `ApiKeyCreateRequest`, `ApiKeyResponse` — deleted.
- `ingestion/kafka/{Log,Replay,SshAudit}*` and `ingestion/clickhouse/{Log,Replay,SshAudit}Writer` — deleted.
- All SSH-encryption plumbing (`seestack.ssh.encryption-key`, `org.apache.sshd:sshd-core`, `spring-boot-starter-websocket`) and their docker env vars — deleted.

3.3 Frontend

```
packages/web/src/features/
├── auth/         LoginPage + RegisterPage
├── overview/     OverviewPage
├── projects/     ProjectsPage + useProjects hook
├── errors/       ErrorsPage + ErrorDetailPage
└── monitors/     MonitorsPage + MonitorDetailPage
```

Key changes:

- **New `/register` route and RegisterPage** (name + email + password).
- **New `/projects` route and ProjectsPage** with list, create form, "Select" button, and an "Ingest key" panel that reveals the raw key once right after creation.
- **Errors now use a dedicated page at `/errors/:fingerprint`** — the `ErrorDetailSheet` drawer and `BulkActionsBar` components were deleted.
- **Sidebar** = `Overview · Projects · Errors · Monitors`.
- **`useAuth` / `useLogin`** rewritten to work with the user-only model (no org fetch, no project switcher based on org).
- `AppShell` auto-selects the first project after login.
- `@seestack/shared`'s `Organization` type, `Plan` type, and every `org` field on `User` / `AuthState` are gone.

3.4 Storage

- **PostgreSQL 16** — the six tables above.
- **ClickHouse 24** — `seestack.errors` (30-day TTL) and `seestack.monitor_checks`.
- **Kafka** — `seestack.errors` and `seestack.monitor-checks` topics only.
- **Redis** — cache.

3.5 Flows

```
SDK ──POST /ingest/v1/errors (X-SeeStack-Key)──▶ ApiKeyAuthFilter
                                                        │
```

```
                                  fingerprint = SHA-256(class + top-5 frames)
                                                        │
                                    ▼ Kafka "seestack.errors"
                                     consumer → ClickHouse seestack.errors
                                              → PostgreSQL error_groups
                                                        ▲
                       Dashboard ──Bearer JWT──▶ JwtAuthFilter → /api/v1/errors

Dashboard ──POST /api/v1/monitors (Bearer JWT)──▶ JwtAuthFilter
                                                        │
                                                 monitor_configs (PG)
                                                        ▲
                                                MonitorScheduler @60s → Kafka
                                                        → ClickHouse seestack.monitor check
```

---

4. Key Features

4.1 Auth (internal)

- `POST /api/auth/register` → BCrypts password, creates user + a "My First Project" with a default API key, returns a JWT.
- `POST /api/auth/login` → validates credentials, returns JWT.
- No OAuth, no email verification, no forgot-password, no SSO.

4.2 Projects

- `GET /api/v1/projects` — list my projects (with `apiKeyPrefix`).
- `POST /api/v1/projects` `{name, platform?}` — returns the created project **and** the raw API key (one-time reveal).
- `GET /api/v1/projects/{id}` — detail with key prefix.
- `DELETE /api/v1/projects/{id}` — cascades to monitors / errors / api_keys.

4.3 Error monitoring

- Ingest endpoint returns 202 immediately.
- Stable SHA-256 fingerprint groups repeats in `error_groups`.
- `GET /api/v1/errors` — list with filters (status, level, env).
- `GET /api/v1/errors/{fingerprint}` — full detail (title, stack trace, timestamps, occurrences, context).
- Dashboard row click → dedicated `/errors/:fingerprint` page with back link, status badges, occurrence count, project badge, first/last seen, stack trace, and Mark-resolved / Ignore actions.

4.4 Website monitoring

- CRUD on `/api/v1/monitors`.

- Scheduler every 60 s; up iff `2xx/3xx ∧ < 30 s`.
- List enriched with current status, last response time, uptime %.

---

5. Contributions / Changes Shipped

- **Removed the organization concept everywhere.** Entity, repo, service, controller, DTOs, migrations, `org_id` columns and FKs, Flyway history, shared TypeScript types, auth store, Zustand hydration, JWT `orgId` claim. Confirmed with `grep -rin "organization\|orgId\|\borg\b" backend/src/main frontend/src` → no matches in kept code.
- **Consolidated 30+ migrations** into a single clean `V1__schema.sql` with only the six tables in scope.
- **Added RegisterPage** and a `/register` route.
- **Added ProjectsPage** with real data, project creation, and the one-time API-key reveal.
- **Replaced the error drawer** with a full `/errors/:fingerprint` page; deleted `ErrorDetailSheet.tsx` and `BulkActionsBar.tsx`.
- **Trimmed backend dependencies** (`spring-boot-starter-websocket`, `sshd-core`) and properties (`seestack.ssh.encryption-key`, `seestack.base-url`, `seestack.backend-url`).

5.1 SDK Setup — added in this iteration

A new dashboard page guides developers through embedding the ingest SDK:

- Route: `/sdk-setup`. Sidebar entry: **SDK Setup** (5th and final item, after Overview / Projects / Errors / Monitors).
- Pulls real projects from `GET /api/v1/projects` and lets the user pick one.
- Shows the project's **ingest key** (the raw `ask_live_…` if the project was just created in this session, otherwise the safe `ask_live_` prefix with a link to create a new key).
- Three language tabs — **JavaScript / Java / Python** — each with three numbered steps: install, initialise with the ingest key, capture an exception. Every snippet includes the real endpoint (`{host}/ingest/v1/errors`), the `X-SeeStack-Key` header, and the actual project key if revealed.
- Copy buttons on every code block.

The SDK files themselves live under `sdks/`:

Language	File
JavaScript	`sdks/javascript/seestack-sdk.js` → class `SeeStack`
Java	`sdks/java/SeeStack.java` → class `SeeStack`
Python	`sdks/python/seestack_sdk.py` → class `SeeStack`

All three are zero-dependency (stdlib only), and all three take the ingest key + endpoint at construction. No Keycloak. No OAuth. No user JWT. No organization context on the ingest path.

Legacy SDK cleanup

- `packages/allstak-js/` — deleted.
- `packages/allstak-python/` — deleted.
- Mixed `allstak` / Keycloak branding in remaining SDK docs, examples, and READMEs — rewritten.
- `backend/src/main/java/com/seestack/shared/security/RequestContext.java` and `RequestContextFilter.java` — deleted (they still carried a `keycloakId` field and were no longer referenced).
- `packages/web/.env.example` with `VITE_KEYCLOAK_*` vars — deleted.
- `WebConfig.java` trimmed to an empty shell.

6. Runtime Validation (Real, this run)

6.1 Backend

- `docker compose up` → 4 infra containers healthy.
- `java -jar …/seestack-backend-0.0.1-SNAPSHOT.jar` → `GET /actuator/health` = `{"status":"UP"}`.
- `\dt` on `seestack`:

  ```
  api_keys · error_groups · flyway_schema_history ·
  monitor_configs · projects · users
  ```

  — no organizations, no billing, no subscriptions, no project_members, no chat, no ssh_servers, no alert_rules, no feature_flags, no notification_log.
- `POST /api/auth/register {email, password, name}` → **201**, returns JWT + auto-created `My First Project`.
- `POST /api/auth/login` → **200**, returns fresh JWT.
- `GET /api/v1/projects` (Bearer) → list of 1 project with `apiKeyPrefix: "ask_live"`.
- `POST /api/v1/projects {name:"E2E Test"}` → **201**, returns the raw `apiKey: "ask_live_…"` **once**.
- SDK `sdks/examples/example-app.js` with that key → 3 × **202**; three rows in PostgreSQL `error_groups` and three rows in ClickHouse `seestack.errors`.
- `POST /api/v1/monitors` twice (Bearer JWT) — Example UP (`https://example.com`) and Dead DOWN (`http://127.0.0.1:1/nope`).
- After scheduler fired, `seestack.monitor_checks` in ClickHouse has one UP (status 200) and one DOWN (status 0).

6.2 Frontend (Chrome MCP)

1. `/register` — filled `Ada Lovelace / ada@example.com / supersecret123`, clicked **Create account** → redirect to `/overview`, token stored.
2. **Sidebar** has exactly: `/overview`, `/projects`, `/errors`, `/monitors`. No other entries.
3. `/projects` — lists both projects (`My First Project` selected, `E2E Test`), shows `INGEST KEY` `ask_live_…` row for each, screenshot captured.
4. Selected `E2E Test` via the **Select** button.
5. `/errors` — table renders 3 real issue rows (1 SyntaxError staging, 2 TypeErrors production).
6. Clicked a row → navigated to `/errors/693c9dff33ea768423ec047d243ffe0dd413461543aaa59a59e5a579cd33cd03`. Full detail page shows: fingerprint preview, `SyntaxError`, the full message, badges (`Unresolved · staging · warning · 1 occurrence · project: E2E Test`), first/last seen, **Mark resolved** / **Ignore** buttons. **No drawer.** Screenshot captured.
7. `/monitors` — renders both monitors, `OPERATIONAL` (`example.com`) and `DOWN` (`127.0.0.1:1`).
8. Text scan of `/monitors` innerText:
   - `organization` / `\borg\b` → **not present**
   - `billing` / `subscription` / `pricing` / `upgrade` / `plan` → **not present**

6.3 Summary matrix

Check	Result
No `organizations` / `plan` / `billing_*` / `subscription`	✅
Backend boots clean, 6 tables only	✅
`/api/auth/register` + `/api/auth/login` return JWT	✅
`/api/v1/projects` list + create + one-time api-key reveal	✅
Monitor scheduler writes UP/DOWN to ClickHouse	✅
Sidebar: Overview, Projects, Errors, Monitors, **SDK Setup**	✅
`/errors/:fingerprint` is a full page (no drawer)	✅
`/sdk-setup` page renders with project picker + 3 languages	✅
JS SDK example (`sdks/examples/example-app.js`) → 3× 202	✅
Python SDK (`sdks/python/seestack_sdk.py`) → 202	✅
Java SDK (`sdks/java/SeeStack.java`) → 202	✅
All 5 events visible on `/errors` after ingest	✅
No `keycloak` / `allstak` / `oauth` anywhere in the UI	✅